

MOVIEMOVIE

Documentation technique

Hyunah Park · Ismail Aabiquella · Ylian Bertrand — Projet SUPINFO, 2025-2026

Table des matières

1. Aperçu du projet.....	3
2. Prérequis et installation	3
2.1 Logiciels requis	3
2.2 Obtention de la clé API TMDB	3
2.3 Obtention des identifiants Google OAuth	3
2.4 Obtention des identifiants Supabase	4
2.5 Configuration des fichiers .env	4
3. Guide de déploiement	5
3.1 Déploiement avec Docker Compose (recommandé)	5
3.2 Architecture des services Docker	5
3.3 Initialisation de la base de données	5
3.4 Déploiement local pour le développement.....	5
3.5 Application mobile (Expo)	6
3.6 Mise en production (Vercel, Railway, Supabase)	6
4. Justification des choix technologiques	7
4.1 Node.js et Express 5	7
4.2 PostgreSQL 16	7
4.3 React 19 et Vite	7
4.4 React Native et Expo 54	7
4.5 JWT et Refresh Tokens	7
4.6 API TMDB.....	7
4.7 Google OAuth 2.0	8
4.8 Supabase (base PostgreSQL managée et stockage)	8
4.9 Docker Compose	8
4.10 Swagger UI (documentation de l'API).....	8
5. Diagrammes UML et schéma relationnel	8
5.1 Diagramme de cas d'utilisation	8
5.2 Diagramme de séquence : interaction avec l'API tierce (TMDB).....	9
5.3 Schéma relationnel de la base de données	10
6. Tables base de données	12
6.1 Table users	12
6.2 Table refresh_tokens.....	12
6.3 Table oauth_codes	12

6.4 Table media_cache.....	12
6.5 Table reviews.....	13
6.6 Table collections.....	13
6.7 Table follows	13
6.8 Table notifications	13
6.9 Table messages	14
6.10 Table activity_log.....	14
6.11 Tables de modération	14
7. Architecture générale.....	14
7.1 Vue d'ensemble.....	14
7.2 Communication entre les couches	15
7.3 Flux d'authentification.....	15
7.4 Flux OAuth Google.....	15
7.5 Cache TMDB	16
8. Sécurité.....	16
8.1 Authentification et tokens.....	16
8.2 OAuth et codes à usage unique.....	16
8.3 Variables d'environnement	16
8.4 Validation des entrées.....	16
8.5 Stockage mobile	16

1. Aperçu du projet

MOVIEMOVIE est une plateforme de gestion et de découverte de contenus cinématographiques et séries télévisées. Elle permet aux utilisateurs de rechercher des films et séries, de gérer leur bibliothèque personnelle, de rédiger des critiques, et d'interagir avec d'autres membres.

Le thème choisi est le cinéma et les séries télévisées. L'application repose sur la base de données The Movie Database (TMDb) comme source principale de données sur les œuvres.

Le projet se compose de trois sous-applications distinctes :

- Une API backend Node.js/Express qui centralise la logique métier.
- Une application web React destinée aux navigateurs.
- Une application mobile React Native/Expo pour iOS et Android.

Les trois applications partagent la même API et la même base de données PostgreSQL.

Le code source du projet est hébergé sur le dépôt Git suivant :

<https://github.com/hyunahparc/supcontent-final-project> (dépôt privé jusqu'à la date de rendu, rendu public ensuite).

Pour consulter la version déployée du projet, l'application web est disponible à l'adresse suivante : <https://moviemovie.space>

La documentation Swagger de l'API est également disponible ici :

<https://moviemovie.up.railway.app/api-docs>

2. Prérequis et installation

2.1 Logiciels requis

- Docker Desktop 4.x ou supérieur (inclut Docker Compose v2)
- Git
- Node.js 20 LTS (pour le développement local uniquement)
- Un compte TMDb pour obtenir une clé API gratuite

2.2 Obtention de la clé API TMDb

1. Créer un compte sur <https://www.themoviedb.org>
2. Accéder aux paramètres du compte, rubrique API.
3. Demander une clé API (type Developer).
4. Copier la valeur du champ API Key (v3 auth).

2.3 Obtention des identifiants Google OAuth

1. Ouvrir Google Cloud Console.

2. Créer un projet ou en sélectionner un existant.
3. Activer les API nécessaires à l'authentification Google OAuth2.
4. Dans Identifiants, créer un ID client OAuth 2.0 de type Application Web.
5. Ajouter ensuite les URI de redirection autorisés correspondant aux environnements local et production (voir section Déploiement).
6. Copier le Client ID et le Client Secret.

2.4 Obtention des identifiants Supabase

1. Créer un compte sur <https://supabase.com> puis créer un nouveau projet.
2. Dans le tableau de bord du projet, ouvrir Project Settings, rubrique Data API.
3. Copier la valeur Project URL : elle correspond à la variable SUPABASE_URL.
4. Ouvrir la rubrique API Keys et copier la clé service_role (secrète) : elle correspond à SUPABASE_SERVICE_KEY.
5. Dans la section Storage, créer un bucket public nommé avatars destiné aux images de profil.
6. Project Settings, rubrique Database : copier la chaîne de connexion (Connection string). Elle sert de DATABASE_URL au backend (base PostgreSQL managée, en développement comme en production).

La clé service_role dispose de droits étendus : elle ne doit jamais être exposée côté client ni committée dans le dépôt (voir section 8.3).

2.5 Configuration des fichiers .env

Le projet utilise trois fichiers .env distincts, un par environnement d'exécution. Chacun se crée en copiant le fichier .env.example correspondant puis en renseignant ses valeurs. Les vrais fichiers .env sont exclus du dépôt par le .gitignore (voir section 8.3).

- .env (racine du projet) — lu par docker-compose.yml pour substituer les variables des trois services ; utilisé uniquement en mode Docker :

```
POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB
TMDB_API_KEY=<votre_cle_tmdb>
JWT_SECRET=<chaîne_aleatoire_longue>
GOOGLE_CLIENT_ID=<...> / GOOGLE_CLIENT_SECRET=<...>
GOOGLE_CALLBACK_URL=http://localhost:3000/api/auth/google/callback
GOOGLE_MOBILE_CALLBACK_URL=<backend_public_https>/api/auth/google/callback
WEB_URL=http://localhost:5173
SUPABASE_URL=<votre_url> / SUPABASE_SERVICE_KEY=<votre_cle>
```

- supcontent-backend/.env — lu par le backend via dotenv lors d'un lancement local (npm run dev) :

```
DATABASE_URL=<chaîne_de_connexion_postgres>
PGSSLMODE=disable (base locale ; à omettre avec Supabase)
```

```
JWT_SECRET / TMDB_API_KEY  
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET / GOOGLE_CALLBACK_URL /  
GOOGLE_MOBILE_CALLBACK_URL  
WEB_URL / SUPABASE_URL / SUPABASE_SERVICE_KEY
```

- supcontent-mobile/.env — lu par Expo :

```
EXPO_PUBLIC_API_URL=<url_du_backend>/api (doit se terminer par /api)
```

Le client web n'utilise pas de fichier .env : en développement, le serveur Vite relaie automatiquement les requêtes /api vers le backend.

3. Guide de déploiement

3.1 Déploiement avec Docker Compose (recommandé)

Cette méthode utilise uniquement le fichier .env à la racine et démarre ensemble la base de données, le backend et le client web.

1. Cloner le dépôt : `git clone https://github.com/hyunahparc/supcontent-final-project.git`
2. Se placer à la racine : `cd supcontent-final-project`
3. Copier le modèle d'environnement : `cp .env.example .env`, puis renseigner `TMDB_API_KEY`, `JWT_SECRET`, `POSTGRES_*`, etc. (voir section 2.5).
4. Construire et démarrer les trois services : `docker compose up`
5. Vérifier l'état des conteneurs : `docker compose ps`
6. Ouvrir le client web sur `http://localhost:5173` (point d'entrée de l'application).
7. API backend : `http://localhost:3000` — documentation Swagger : `http://localhost:3000/api-docs`.
8. Le schéma de la base est initialisé automatiquement depuis `supcontent-backend/db/cine.sql` au premier démarrage.

3.2 Architecture des services Docker

Service	Image	Port exposé	Description
supcontent_db	postgres:16-alpine	Interne seulement	Base de données PostgreSQL
supcontent_backend	node:20-alpine	3000	API Express
supcontent_web	node:20-alpine + nginx	5173	Application React servie par Nginx

3.3 Initialisation de la base de données

Au premier démarrage, Docker Compose monte le fichier `supcontent-backend/db/cine.sql` dans le répertoire d'initialisation de PostgreSQL. Ce fichier crée automatiquement toutes les tables, index, et contraintes.

3.4 Déploiement local pour le développement

Chaque partie se lance séparément et lit son propre fichier .env (et non celui de la racine).

Backend — utilise supcontent-backend/.env :

1. cd supcontent-backend
2. npm install
3. cp .env.example .env, puis définir DATABASE_URL (base PostgreSQL locale avec PGSSLMODE=disable, ou chaîne de connexion d'un projet Supabase), JWT_SECRET, TMDB_API_KEY, etc.
4. Pour une base locale, charger le schéma : psql -d supcontent -f db/cine.sql
5. npm run dev → http://localhost:3000 (nodemon)

Client web — aucun fichier .env nécessaire (le serveur Vite relaie /api vers le backend) :

1. cd supcontent-web
2. npm install
3. npm run dev → http://localhost:5173

3.5 Application mobile (Expo)

Le client mobile s'exécute toujours via Expo, que le backend ait été lancé via Docker ou en local. Il lit supcontent-mobile/.env.

1. cd supcontent-mobile
2. npm install
3. cp .env.example .env, puis définir EXPO_PUBLIC_API_URL (doit se terminer par /api).
4. npx expo start, puis scanner le QR code avec l'application Expo Go (iOS / Android).

Valeur de EXPO_PUBLIC_API_URL selon le besoin de connexion Google sur mobile :

- Avec connexion Google : utiliser une URL de backend publique en HTTPS (backend déployé ou tunnel ngrok), ex. https://votre-hote-public/api, et renseigner le GOOGLE_MOBILE_CALLBACK_URL correspondant dans la Google Cloud Console.
- Sans connexion Google (email/mot de passe) : utiliser l'IP LAN Wi-Fi de la machine, ex. http://<ip-wifi>:3000/api. La connexion Google ne fonctionne pas dans ce cas.

3.6 Mise en production (Vercel, Railway, Supabase)

L'application est déployée en production et accessible publiquement :

- Client web : hébergé sur Vercel avec le domaine personnalisé moviemovie.space ; chaque requête /api/* est relayée vers le backend.
- Backend : hébergé sur Railway à l'adresse https://moviemovie.up.railway.app (documentation Swagger sur /api-docs).
- Base de données : PostgreSQL managé sur Supabase ; Supabase Storage héberge également les avatars.

Le client web et le backend sont redéployés automatiquement à chaque push sur GitHub (intégration Vercel et Railway). Docker Compose reste dédié à l'installation locale en une commande.

URLs en production — Web : <https://moviemovie.space>

Backend API : <https://moviemovie.up.railway.app>

Documentation API : <https://moviemovie.up.railway.app/api-docs>

4. Justification des choix technologiques

4.1 Node.js et Express 5

Node.js est choisi pour sa performance sur les opérations d'entrées/sorties non bloquantes, ce qui convient aux requêtes parallèles vers TMDb et la base de données. Express 5 apporte une gestion native des promesses dans les routes, éliminant le besoin de blocs try/catch explicites.

4.2 PostgreSQL 16

PostgreSQL est choisi pour sa robustesse, son support natif du type JSONB (utilisé pour le cache TMDb), et ses contraintes relationnelles strictes. La colonne full_data de type JSONB permet de stocker les données brutes TMDb sans schéma fixe tout en restant interrogeable.

4.3 React 19 et Vite

React est le framework frontal le plus répandu, avec un écosystème mature. Vite remplace Create React App pour des temps de build et de démarrage significativement plus courts. React Router gère la navigation côté client et le client web communique avec le backend via Axios.

4.4 React Native et Expo 54

React Native permet de réutiliser une logique similaire à celle du client web, notamment dans la structure des appels API et de l'authentification. Expo simplifie la configuration native et fournit Expo Router, un système de routage basé sur les fichiers similaire à Next.js. expo-secure-store chiffre les tokens sur l'appareil.

4.5 JWT et Refresh Tokens

L'authentification sans état par JWT (15 minutes) réduit les requêtes à la base de données. Les refresh tokens (30 jours) stockés en base sous forme de hash SHA-256 permettent la révocation serveur, ce qui est impossible avec des JWT seuls.

4.6 API TMDb

TMDb est la principale API cinématographique gratuite avec une couverture mondiale, des données structurées (casting, bande-annonce, genres), et des limites de taux généreuses. Le cache JSONB en base réduit les appels API et améliore les temps de réponse.

4.7 Google OAuth 2.0

Google OAuth offre une connexion rapide sans création de mot de passe. La stratégie `passport-google-oauth20` simplifie l'intégration. Un système de codes à usage unique (OTP) sécurise le transfert de tokens vers le client après la redirection OAuth.

4.8 Supabase (base PostgreSQL managée et stockage)

Supabase fournit deux services au projet. D'une part, une base de données PostgreSQL managée, utilisée en production (et possible en développement), accessible via une chaîne de connexion standard. D'autre part, Supabase Storage héberge les avatars des utilisateurs, avec une API compatible S3, une CDN intégrée et un niveau gratuit suffisant pour un projet de cette échelle.

4.9 Docker Compose

Docker Compose garantit la reproductibilité de l'environnement sur toute machine. Les trois services (base de données, backend, frontend) sont déclarés dans un seul fichier, avec healthchecks et réseau interne isolé.

4.10 Swagger UI (documentation de l'API)

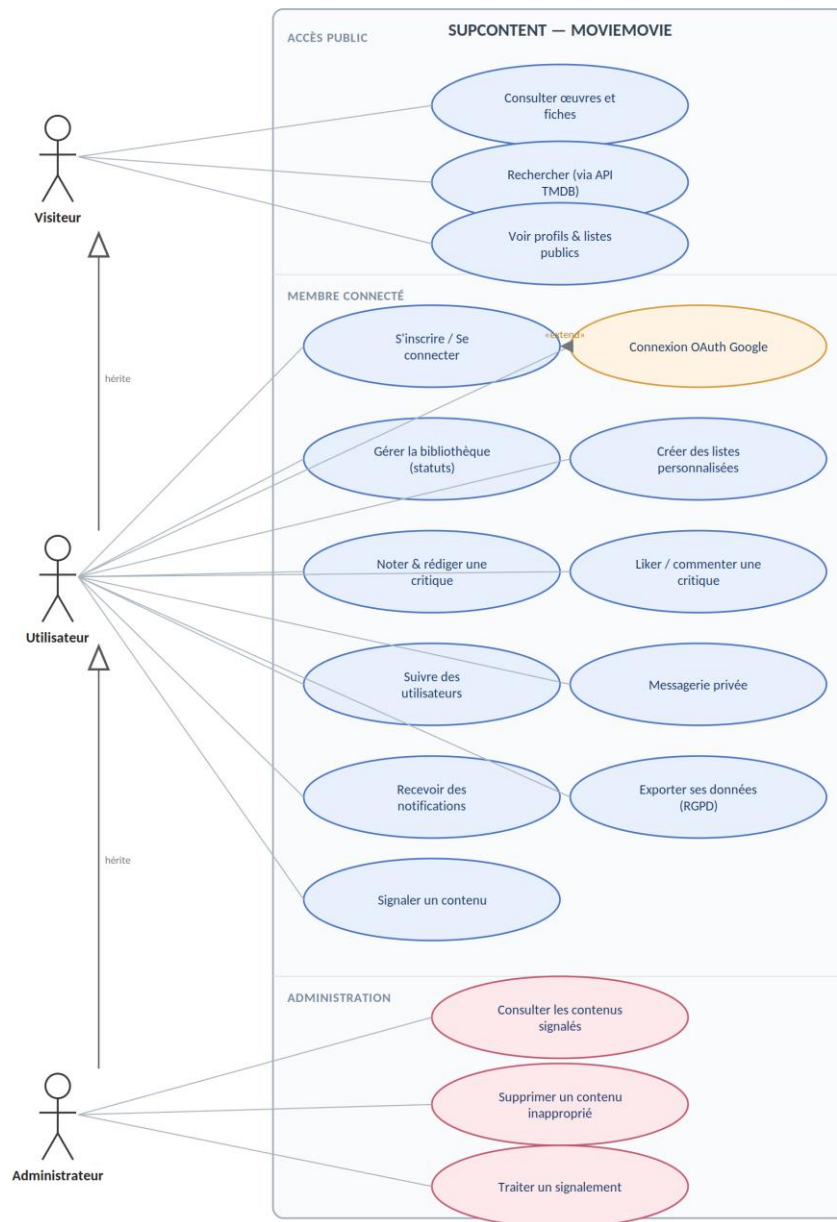
La documentation de l'API REST est générée à partir d'une spécification OpenAPI (fichier `swagger.yaml`) et exposée via Swagger UI sur l'endpoint `/api-docs`. Elle permet d'explorer et de tester chaque route du backend directement depuis le navigateur.

5. Diagrammes UML et schéma relationnel

Cette section présente les modèles du projet : le diagramme de cas d'utilisation, le diagramme de séquence de l'interaction avec l'API tierce TMDB, et le schéma relationnel de la base de données.

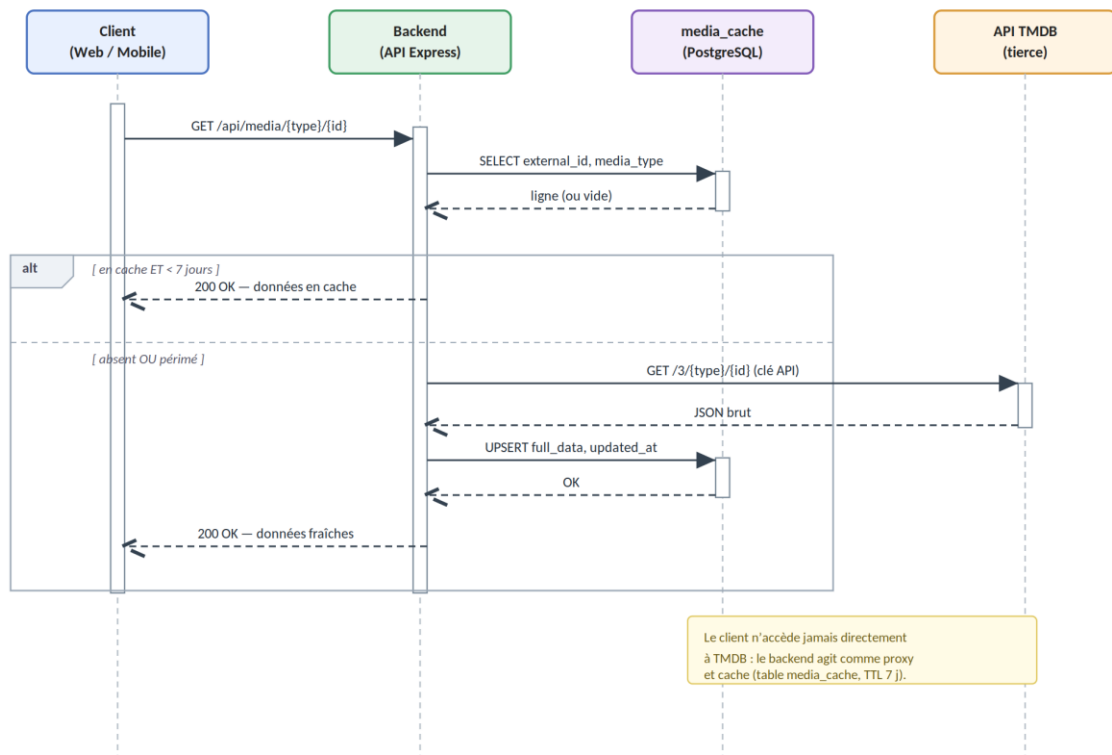
5.1 Diagramme de cas d'utilisation

Le diagramme distingue trois acteurs. Le Visiteur, non authentifié, dispose d'un accès en consultation seule. L'utilisateur hérite des droits du Visiteur et ajoute l'ensemble des interactions sociales et de gestion de bibliothèque. L'Administrateur hérite de l'utilisateur et dispose en plus des fonctions de modération.



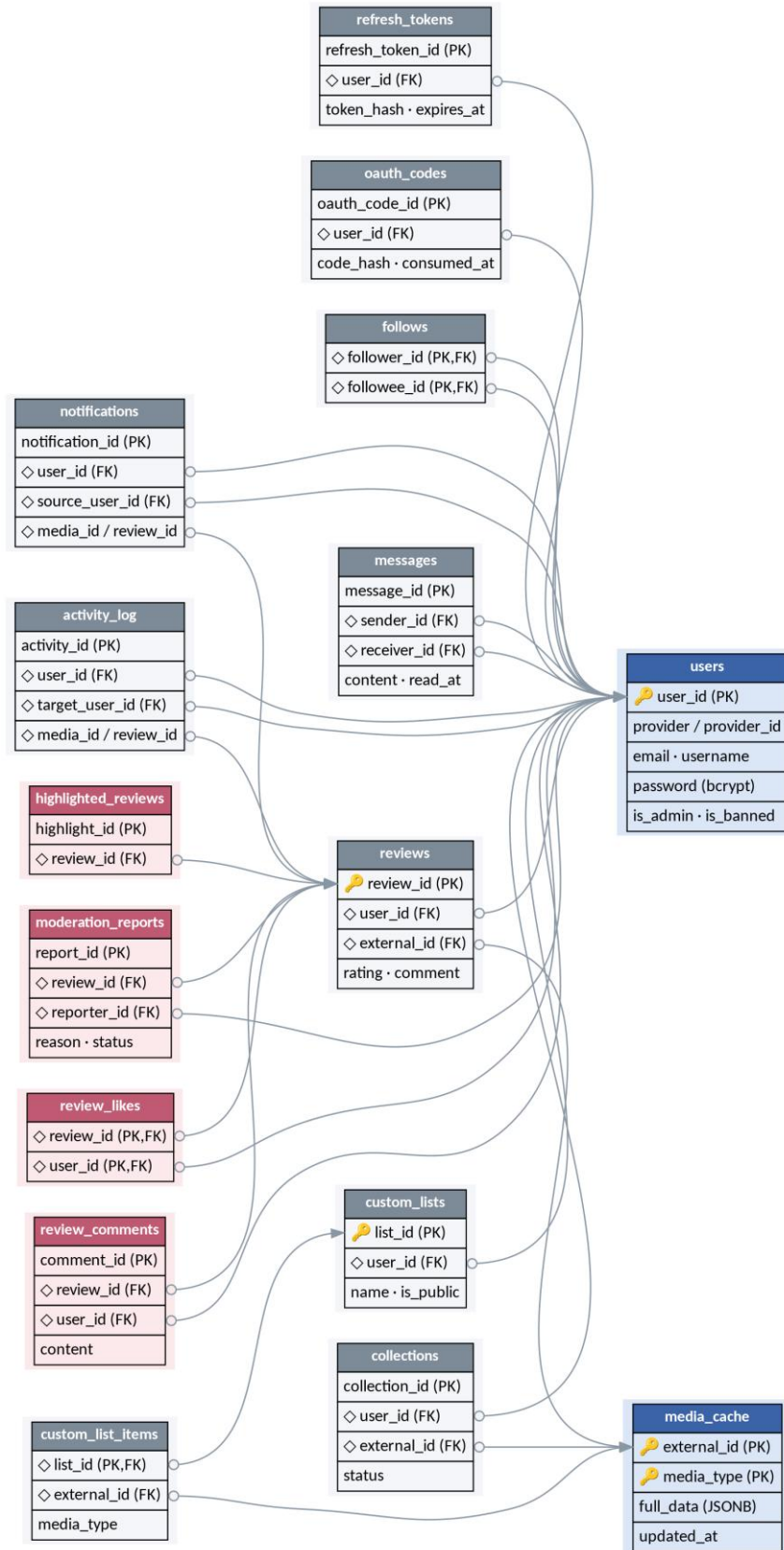
5.2 Diagramme de séquence : interaction avec l'API tierce (TMDB)

Ce diagramme illustre la stratégie de cache décrite en section 7.5. Le client n'interroge jamais TMDB directement : le backend agit comme proxy. Il sert la donnée depuis la table `media_cache` si elle a moins de sept jours, sinon il interroge TMDB, met le cache à jour par UPSERT, puis répond au client.



5.3 Schéma relationnel de la base de données

Le schéma ci-dessous présente les tables, leurs clés primaires (PK), leurs clés étrangères (FK) et les relations entre elles. Les tables users et media_cache (en bleu) constituent les deux entités centrales ; les tables de modération et d'interaction sociale sont regroupées en rose.



6. Tables base de données

6.1 Table users

Colonne	Type	Contraintes	Description
user_id	SERIAL	PK	Identifiant unique
provider	VARCHAR(20)	DEFAULT 'local'	Fournisseur : local ou google
provider_id	VARCHAR(255)	NULL	Identifiant Google
email	VARCHAR(255)	UNIQUE(email, provider)	Adresse email
username	VARCHAR(50)		Nom d'utilisateur
password	VARCHAR(255)	NULL	Hash bcrypt
avatar	VARCHAR(255)	NULL	URL Supabase
bio	TEXT	NULL	Biographie
link	VARCHAR(255)	NULL	Lien externe
preferred_language	VARCHAR(5)	DEFAULT 'en'	Langue préférée
is_admin	BOOLEAN	DEFAULT FALSE	Rôle administrateur
is_banned	BOOLEAN	DEFAULT FALSE	Compte banni / désactivé
created_at	TIMESTAMPTZ	DEFAULT NOW()	Date de création

6.2 Table refresh_tokens

Colonne	Type	Contraintes	Description
refresh_token_id	SERIAL	PK	Identifiant
token_hash	CHAR(64)	UNIQUE	Hash SHA-256 du token
user_id	INT	FK users CASCADE	Propriétaire
expires_at	TIMESTAMPTZ		Date d'expiration
revoked_at	TIMESTAMPTZ	NULL	Date de révocation
created_at	TIMESTAMPTZ	DEFAULT NOW()	Date de création

6.3 Table oauth_codes

Colonne	Type	Contraintes	Description
oauth_code_id	SERIAL	PK	Identifiant
code_hash	CHAR(64)	UNIQUE	Hash SHA-256 du code
user_id	INT	FK users CASCADE	Utilisateur concerné
expires_at	TIMESTAMPTZ		Expiration (5 min)
consumed_at	TIMESTAMPTZ	NULL	Date de consommation
created_at	TIMESTAMPTZ	DEFAULT NOW()	Date de création

6.4 Table media_cache

Colonne	Type	Contraintes	Description
external_id	BIGINT	PK partielle	ID TMDB
media_type	VARCHAR(10)	PK partielle	movie / series
full_data	JSONB	NOT NULL	Données TMDB

			complètes
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Dernière mise à jour

6.5 Table reviews

Colonne	Type	Contraintes	Description
review_id	SERIAL	PK	Identifiant
user_id	INT	FK users SET NULL	Auteur
external_id	BIGINT	FK media_cache	ID TMDB du média
media_type	VARCHAR(10)		movie / series
rating	NUMERIC(2,1)	0.5 à 5.0	Note
comment	TEXT	NULL	Texte de la critique
created_at	TIMESTAMPTZ	DEFAULT NOW()	Date de création
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Dernière modification

6.6 Table collections

Colonne	Type	Contraintes	Description
collection_id	SERIAL	PK	Identifiant
user_id	INT	FK users CASCADE	Propriétaire
external_id	BIGINT	FK media_cache	ID TMDB
media_type	VARCHAR(10)		movie / series
status	VARCHAR(20)		To Watch / Watching / Completed / Dropped
created_at	TIMESTAMPTZ	DEFAULT NOW()	Date d'ajout

6.7 Table follows

Colonne	Type	Contraintes	Description
follower_id	INT	PK, FK users CASCADE	Utilisateur qui suit
followee_id	INT	PK, FK users CASCADE	Utilisateur suivi

6.8 Table notifications

Colonne	Type	Contraintes	Description
notification_id	SERIAL	PK	Identifiant
user_id	INT	FK users CASCADE	Destinataire
type	VARCHAR(20)		like, comment ou follow
source_user_id	INT	FK users SET NULL	Émetteur
media_id	BIGINT	FK media_cache, NULL	Média concerné
media_type	VARCHAR(10)	NULL	movie / series
review_id	INT	FK reviews CASCADE, NULL	Critique concernée
is_read	BOOLEAN	DEFAULT FALSE	Statut de lecture
created_at	TIMESTAMPTZ	DEFAULT NOW()	Date de création

6.9 Table messages

Colonne	Type	Contraintes	Description
message_id	SERIAL	PK	Identifiant
sender_id	INT	FK users CASCADE	Expéditeur
receiver_id	INT	FK users CASCADE	Destinataire
content	TEXT	NOT NULL	Contenu (max 1000 car.)
read_at	TIMESTAMPTZ	NULL	Date de lecture
created_at	TIMESTAMPTZ	DEFAULT NOW()	Date d'envoi

6.10 Table activity_log

Colonne	Type	Contraintes	Description
activity_id	SERIAL	PK	Identifiant
user_id	INT	FK users CASCADE	Auteur de l'activité
activity_type	VARCHAR(20)		review ou collection
target_user_id	INT	FK users SET NULL, NULL	Utilisateur cible
media_id	BIGINT	FK media_cache, NULL	Média concerné
media_type	VARCHAR(10)	NULL	movie / series
review_id	INT	FK reviews CASCADE, NULL	Critique associée
metadata	JSONB	DEFAULT '{}'	Données supplémentaires
created_at	TIMESTAMPTZ	DEFAULT NOW()	Date

6.11 Tables de modération

Table	Colonnes principales	Description
moderation_reports	report_id, review_id, reporter_id, reason, status, created_at	Signalements de critiques
highlighted_reviews	highlight_id, review_id, created_at	Critiques mises en avant par l'admin
review_likes	review_id + user_id (PK composite)	J'aime sur les critiques
review_comments	comment_id, review_id, user_id, content, created_at	Commentaires sur les critiques
custom_lists	list_id, user_id, name, is_public, created_at, updated_at	Listes personnalisées
custom_list_items	list_id + external_id + media_type (PK), added_at	Éléments d'une liste

7. Architecture générale

7.1 Vue d'ensemble

L'application suit une architecture trois-tiers classique :

- Couche présentation : application web (React) et application mobile (React Native/Expo).
- Couche métier : API REST Express sur Node.js.
- Couche données : base de données PostgreSQL.

Côté serveur, chaque requête traverse les couches suivantes : routes → middlewares (authentification, validation, limitation de débit) → contrôleurs → services (TMDB, notifications, activité, refresh tokens) → base de données.

7.2 Communication entre les couches

Les deux clients (web et mobile) communiquent exclusivement avec le backend via HTTP REST. Le backend ne retourne jamais de HTML ; il répond en JSON.

Le backend joue le rôle de proxy pour l'API TMDB : le client n'a jamais accès direct à TMDB et n'a pas connaissance de la clé API. Toutes les requêtes TMDB passent par le backend, qui met le résultat en cache dans la table `media_cache`.

7.3 Flux d'authentification

1. Le client envoie les identifiants à POST `/api/auth/login`.
2. Le backend vérifie le mot de passe avec `bcrypt` et génère un access token (15 min) et un refresh token (30 jours).
3. Le refresh token est hashé en SHA-256 et stocké en base.
4. Quand l'access token expire, le client appelle POST `/api/auth/refresh` avec le refresh token.
5. Le backend échange l'ancien refresh token contre un nouveau (rotation).
6. À la déconnexion, le refresh token est révoqué en base (`revoked_at` renseigné).

7.4 Flux OAuth Google

1. Le client redirige vers GET `/api/auth/google`.
2. Passport redirige l'utilisateur vers la page de consentement Google.
3. Après approbation, Google rappelle `/api/auth/google/callback`.
4. Le backend crée ou retrouve l'utilisateur, puis génère un code à usage unique (5 min, hash SHA-256).
5. Le backend redirige ensuite :
 - le client web vers `${WEB_URL}/oauth/callback?code=...`
(par exemple `http://localhost:5173/oauth/callback?code=...` en local ou `https://moviemovie.space/oauth/callback?code=...` en production) ;
 - le client mobile vers le deep link de l'application, par exemple `supcontent://oauth/callback?code=...`
6. Le client échange le code contre un token pair via POST `/api/auth/oauth/exchange`.
7. Le code est marqué comme consommé et ne peut plus être réutilisé.

7.5 Cache TMDB

À chaque requête sur une fiche média, le backend vérifie si la donnée est présente en base et si elle a moins de 7 jours. Si oui, il retourne la donnée en cache. Sinon, il interroge TMDB, transforme la réponse, et met à jour la table `media_cache` avec un UPSERT.

8. Sécurité

8.1 Authentification et tokens

- Les mots de passe sont hashés avec bcrypt (facteur de coût 10). Aucun mot de passe en clair n'est stocké.
- Les access tokens sont des JWT signés avec `JWT_SECRET` et ont une durée de vie de 15 minutes afin de limiter l'exposition en cas de vol.
- Les refresh tokens ne sont pas des JWT : ce sont des chaînes aléatoires opaques. Ils sont stockés en base uniquement sous forme de hash SHA-256. La valeur brute n'est jamais persistée.
- La rotation des refresh tokens invalide l'ancien token à chaque renouvellement et génère un nouveau refresh token.
- La révocation côté serveur permet de déconnecter un utilisateur en invalidant son refresh token, même si celui-ci n'est pas encore expiré.

8.2 OAuth et codes à usage unique

- Après le retour OAuth, le backend génère un code aléatoire court-terme (5 minutes).
- Ce code est hashé avant stockage et marqué `consumed_at` après usage.
- Cette approche évite de faire transiter les JWT dans les redirections d'URL.

8.3 Variables d'environnement

Aucune clé secrète n'est codée en dur dans le code source. Toutes les valeurs sensibles (clés JWT, clés API, identifiants base de données, credentials Google) sont chargées depuis le fichier `.env` via `dotenv`. Le fichier `.env` est exclu du dépôt git par le `.gitignore`.

8.4 Validation des entrées

- Les données d'inscription et de connexion sont validées avec Joi avant tout traitement.
- Le contenu des messages est limité à 1000 caractères.
- Les ratings sont contraints entre 0.5 et 5.0 avec un pas de 0.5.
- Les messages directs sont restreints aux paires d'utilisateurs avec abonnement mutuel.

8.5 Stockage mobile

Sur mobile, les tokens sont stockés dans expo-secure-store, qui utilise le trousseau du système d'exploitation (Keychain sur iOS, EncryptedSharedPreferences sur Android). Les tokens ne sont pas stockés en texte clair.